An Empirical Study of GPU Kernel Performance Gaps in Modern Domain-Specific Languages

Anonymous Author(s)

Abstract

Modern GPU domain-specific languages (DSLs), such as Triton and TileLang, seek to make high-performance kernel development accessible without requiring deep expertise in CUDA or hardware-specific optimization. However, despite their increasing adoption, it remains unclear how closely DSL-written kernels approach the performance of highly optimized vendor libraries and what factors limit that performance.

We present the first empirical study of the performance gap between DSL-written kernels and library implementations across 22 GPU kernels spanning five operator categories: GEMM, attention, convolution, normalization, and element-wise or reduction operators. Our benchmark suite is built from TritonBench, TileLang examples, and additional implementations for missing configurations. Across these workloads, we find that DSL performance is highly uneven across operator classes. Triton reaches 32–58% of library performance on GEMM and 103% on element-wise kernels, but only 35% on convolution. TileLang reaches up to 56% of cuBLAS throughput on large GEMM shapes, yet remains below 10% of cuDNN throughput on convolution. We also observe a severe normalization anamoly in TileLang, where large LayerNorm is 314× slower than PyTorch.

To explain these gaps, we combine end-to-end benchmarking with hardware counter analysis and identify four recurring causes of convolution underperformance: *lack of vectorization for strided memory accesses, mismatch between autotuning search spaces and convolution workloads, register pressure from large-filter accumulations*, and *absence of Winograd-based algorithm support*. Guided by this analysis, we implement targeted fixes that recover 98% of PyTorch throughput on LayerNorm and RMSNorm and improve Triton convolution performance to 80% of cuDNN. The remaining gap is primarily associated with missing Winograd support. Our data is available at https://doi.org/10.5281/zenodo.19337858.

Keywords

GPU kernels, domain-specific languages, Triton, TileLang, cuBLAS, cuDNN, empirical study, performance analysis, convolution

ACM Reference Format:

Anonymous Author(s). 2026. An Empirical Study of GPU Kernel Performance Gaps in Modern Domain-Specific Languages. In . ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Conference'17, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Deep learning systems are now widely deployed, and their efficiency depends heavily on GPU acceleration [22]. In practice, this acceleration is delivered through a small set of computational kernels that dominate execution time, including dense linear algebra and core neural network primitives [7]. For these operations, current frameworks largely rely on vendor libraries such as cuBLAS and cuDNN [3, 17], which have long served as the main source of high-performance GPU execution.

This execution model is becoming insufficient for modern workloads. As model architectures grow more complex, they increasingly require fused operators, specialized kernels, and model-specific computations that extend beyond the fixed functionality exposed by vendor libraries [26]. As a result, achieving high performance now depends not only on the quality of library implementations, but also on the ability to generate efficient custom kernels when no suitable library routine exists. In this setting, performance becomes a primary criterion for evaluating kernel-generation approaches, since the practical value of a custom kernel depends on whether it can approach the efficiency of optimized library code [14].

To address this need, recent work has shifted toward domain-specific languages (DSLs) for GPU kernel generation. Systems such as Triton [18] and TileLang [27] allow developers to express kernels using tile-level abstractions while delegating low-level concerns, such as memory movement, scheduling, and code generation, to the compiler. This approach is attractive because it aims to reduce the cost of kernel development without giving up the performance expected from optimized GPU code. Its importance is no longer limited to research prototypes. Triton, for example, has been integrated into mainstream compilation workflows and now serves as a lowering target in `torch.compile`, which makes the efficiency of DSL-generated kernels a practical concern for modern deep learning software [1].

Existing work suggests that DSL kernel generation illustrates promising results on several operators such as matrix multiplication and attention, which has encouraged the view that tile-based programming can provide both programmability and efficiency [8]. However, the available evidence is concentrated in kernels whose structure aligns naturally with tiling. It does not establish that the same level of efficiency extends to other important operators, especially those with more irregular access patterns or stronger dependence on algorithm selection.

A performance gap in this setting has direct software engineering consequences. A high-level kernel language is useful in practice only if it provides not only expressiveness, but also performance behavior that developers can reason about. When a DSL-generated kernel is functionally correct yet substantially slower than the corresponding library implementation, the developer must determine whether the loss arises from the algorithm, the schedule, or

the compiler. The paper argues that current toolchains offer little diagnostic support for making that distinction, which makes performance deficits difficult to predict and difficult to correct in production workflows.

Convolution is the clearest case in which this problem becomes visible [21]. Existing evidence for DSL efficiency is concentrated in GEMM and attention: Triton's original matrix multiplication results and later attention kernels such as FlashAttention [6] established that tile-based programming can match or approach highly optimized baselines on these workloads. By contrast, convolution support is less mature. The paper notes that practitioners have repeatedly reported substantial gaps between Triton-based convolution kernels and cuDNN, while TileLang's convolution behavior relative to cuDNN had not been systematically evaluated before this study [21]. The result is a concrete gap in knowledge: the field does not yet know how large the convolution gap is, how consistently it appears, what causes it, or how much of it can be reduced through targeted engineering changes.

This paper addresses that problem through an empirical study of performance gaps between DSL-written kernels and optimized libraries. It evaluates Triton and TileLang against cuBLAS and cuDNN on a suite of 21 kernels spanning five operator classes: GEMM, attention, convolution, normalization, and element-wise operations. The study is organized around three questions: the magnitude and distribution of the gap, the factors that cause it, and the extent to which targeted mitigations can reduce it. The evaluation is conducted on an NVIDIA RTX 4000 Ada Generation GPU and is supported by hardware-counter profiling with Nsight Compute.

The results show that DSL efficiency is not uniform across operator classes. Triton is competitive on element-wise kernels and performs well on some normalization workloads, but reaches only 35–49% of cuDNN throughput on the tested convolution configurations. TileLang shows a larger deficit on convolution, below 10% of cuDNN throughput on the evaluated cases, and also exhibits a severe normalization anomaly that the paper traces to reduction and vectorization issues. Root-cause analysis attributes convolution underperformance to four main factors: missing vectorization for strided memory accesses, mismatch in autotuning search space, register pressure for larger filters, and the absence of Winograd algorithm support. Targeted mitigations recover a substantial part of the lost performance, but they do not fully eliminate the convolution gap.

The contribution of this paper is therefore both empirical and practical:

- **Empirical:** First systematic characterization of the performance gap between modern GPU DSLs and optimized libraries across a representative set of kernel classes.
- **Practical:** Identifies where current tile-based DSLs already deliver competitive performance, where they fall short, and which compiler and scheduling limitations are responsible for the remaining deficits.
- **Defining boundaries:** Establishes the current boundary of efficient tile-based programming.
- **Forward-looking:** Provides a concrete basis for improving DSL support for convolution and related workloads.

2 Background

2.1 Tile-Based GPU Programming

A GPU executes thousands of threads organized into *thread blocks*, each occupying a *streaming multiprocessor* (SM) with fast programmer-managed scratchpad (*shared memory*). The roofline model [28] captures the two performance limits: arithmetic throughput (TFLOPS) and memory bandwidth (GB/s); high-performance kernels tile data [29] to maximize shared memory reuse and stay compute-bound.

2.2 Triton

Triton [24] is a Python-embedded DSL and compiler in which the unit of computation is a *tile*: a statically shaped, multi-dimensional array operated on by all threads in a program instance. The Triton compiler performs memory coalescing, shared memory allocation, thread swizzling, and software pipelining automatically, targeting NVIDIA PTX through an MLIR-based compilation pipeline [10]. Triton exposes performance tuning via `@triton.autotune`, sweeping programmer-specified tile configurations and caching the fastest per problem shape. Triton has been integrated as the default lowering target for `torch.compile` since PyTorch 2.0 [1].

Triton's support for convolution is less mature: an im2col-style lowering exists but has been observed to fall substantially short of cuDNN performance on common workloads [21].

2.3 TileLang

TileLang [27] is a DSL developed at Microsoft Research that separates the *algorithm* (what to compute) from the *schedule* (how to map computation onto GPU resources) more explicitly than Triton, allowing the programmer to independently tune memory staging, thread layout, and pipeline depth without changing the functional description. TileLang compiles through Apache TVM's infrastructure with explicit hardware intrinsic support on NVIDIA and AMD architectures. TileLang's convolution support and its performance relative to cuDNN have not been systematically evaluated prior to this work.

2.4 cuBLAS and cuDNN

cuBLAS [17] is NVIDIA's BLAS implementation for CUDA, providing highly optimized routines for dense linear algebra, with `cuBLASLt` offering a flexible GEMM API with configurable layouts and compute types (FP32, FP16, BF16, FP8, INT8). cuBLAS dispatches the fastest GEMM implementation via a runtime heuristic recommender trained on problem shapes and hardware generation. The underlying template library, CUTLASS [15], is publicly available and serves as a reference for DSL performance comparisons.

cuDNN [3, 4] covers the full set of deep learning primitives: convolution (forward and backward passes), pooling, normalization, recurrent layers, and attention. cuDNN selects among implicit GEMM, Winograd (3×3 filters), FFT (large filters), and direct convolution at runtime via `cudnnFind`, benchmarking all candidates per configuration.

2.5 TritonBench

TritonBench [13] is a comprehensive collection of production-grade Triton kernels curated from high-starred open-source repositories,

covering attention variants, GEMM, softmax, normalization, and fused operations. It provides both functional correctness metrics and efficiency metrics — memory bandwidth utilization and GPU efficiency as a fraction of theoretical peak TFLOPS — making it a natural starting point for a performance gap study. Our kernel suite is derived from TritonBench's GitHub channel (184 kernels from 95 repositories), supplemented with TileLang re-implementations and additional convolution configurations.

3 Methodology

We evaluate whether current GPU DSLs can match vendor libraries on representative deep learning operators, and, when they do not, identify the cause of the gap. To this end, we construct a kernel suite that covers the main computation patterns in modern models, compare DSL implementations against library-backed PyTorch baselines under matched configurations, and profile both end-to-end performance and low-level hardware behavior.

3.1 Kernel Suite

Our benchmark suite covers five operator categories that capture the dominant computation patterns in modern deep learning: matrix multiplication, attention, convolution, normalization, and element-wise or reduction operators. In total, the suite contains 22 kernels: three **GEMM** workloads (i.e., dense matrix multiplication, batched matrix multiplication, and fused linear plus activation), one **attention** workload (i.e., scaled dot-product attention / FlashAttention variants), one **convolution** workload (i.e., Conv2d with 1×1 to 7×7 filters, including depthwise and strided cases), two **normalization** workloads (i.e., LayerNorm and RMSNorm), and fifteen **element-wise or reduction** workloads including add, mul, relu, leaky_relu, softmax, log_softmax, logsumexp, swiglu, argmax, max reduction, mean reduction, cross-entropy, matrix transpose, index select, and embedding.

We derive these workloads from TritonBench [13], the TileLang example repository [27], and our own implementations for configurations not covered by either source. We exclude operators with sparse inputs or runtime-dependent output shapes, since they do not admit stable or meaningful library baselines.

3.2 Baseline Construction

For each workload, we compare DSL implementations against the corresponding vendor-library path exposed through PyTorch. GEMM baselines use cuBLAS via `torch.matmul`. Convolution baselines use `nn.Conv2d` in NHWC layout with `allow_tf32=False`. Normalization baselines use `F.layer_norm` and `F.rms_norm`. Element-wise and reduction baselines use standard PyTorch eager execution.

For attention, we use PyTorch scaled dot-product attention with the FlashAttention backend enabled through `enable_flash_sdp(True)`. We report these results separately because the Triton and TileLang implementations are not algorithmically identical to the PyTorch baseline.

3.3 Profiling Setup

We measure both end-to-end latency and hardware performance counters. Latency is used for all comparisons against library baselines, while hardware counters are used to investigate the causes of observed performance gaps.

End-to-end throughput. For each workload and input configuration, we measure GPU execution time using CUDA events (`cudaEventRecord` and `cudaEventElapsedTime`). Each measurement uses 10 warm-up iterations followed by 100 timed iterations, and we report the average over the timed runs. All results reflect GPU-side execution only and exclude host-side dispatch overhead. To reduce measurement noise, we run each workload in isolation without concurrent GPU activity.

Hardware performance counters. To support root-cause analysis, we collect hardware counter profiles with NVIDIA Nsight Compute [16]. We group the collected counters into three classes: *memory access* (e.g., global load/store efficiency, L1/L2 hit rates, coalescing behavior, and vectorized-load fraction), *compute utilization* (e.g., SM occupancy, warp stall cycles, and Tensor Core utilization), and *instruction behavior* (e.g., `cp.async` issue count and register spill). Each profile is collected from a single execution, and we verify stability within 2% across five repeated runs.

3.4 Metrics

Our primary metric is **library efficiency**,

$$E_{\text{lib}} = \frac{t_{\text{lib}}}{t_{\text{DSL}}} \times 100\%,$$

where t_{lib} denotes the latency of the cuBLAS or cuDNN baseline and t_{DSL} denotes the latency of the corresponding DSL implementation under the same hardware and input configuration. A value of 100% indicates parity with the library baseline; lower values indicate that the DSL implementation is slower.

Secondary metrics: throughput ($T = 2 \cdot \text{FLOPs}/t$), hardware efficiency ($E_{\text{hw}} = T/T_{\text{peak}}$), and memory bandwidth utilization ($B = \text{bytes}/(t \cdot B_{\text{peak}})$)—are used qualitatively in root-cause analysis (RQ2) to interpret counter measurements; they are not tabulated separately.

3.5 Experimental Setup

Hardware. All experiments are conducted on an NVIDIA RTX 4000 Ada Generation GPU (Ada Lovelace, `sm_89`) with 20 GB GDDR6 memory, 360 GB/s peak memory bandwidth, and a 48 MB L2 cache. The system uses NVIDIA Driver 595 and CUDA Toolkit 12.6.

Software. Our software stack consists of PyTorch 2.8.0+cu126, Triton 3.4.0, TileLang 0.1.6.post1, cuDNN 9.1.0.2, Python 3.9.25, and Nsight Compute 2024.3.2.0. We pin all versions in a reproducibility manifest distributed with the benchmark suite. For all experiments, we fix random seeds, disable cuDNN benchmark mode (`torch.backends.cudnn.benchmark=False`), and pre-warm the Triton auto-tuner before measurement.

4 Research Questions

We organize the study around three research questions. The first quantifies performance gaps, the second explains them, and the

Table 1: GEMM-family kernels (FP16) latency (ms, lower is better). E_{lib} = library efficiency relative to PyTorch/cuBLAS baseline. Bold marks best DSL result per row.

Configuration	PyTorch	Triton	E_{lib}	TileLang	E_{lib}
Square matmul					
16384^2	114.7	361.9	31.7%	203.3	56.4%
Batched matmul					
64×128^2	0.020	**0.026**	**77.1%**	0.076	26.5%
128×2048^2	3.238	5.564	58.2%	30.32	10.7%
Fused linear+activation					
$1 \times 2048 \rightarrow 16384$	46.7	89.9	52.0%	**23.4**	**199.7%**

third evaluates whether they can be reduced through targeted changes.

RQ1 (Characterization). What performance gap separates Triton and TileLang kernels from cuBLAS and cuDNN baselines, and how does this gap differ across operator categories?

RQ2 (Root-Cause Analysis). What compiler and architectural factors contribute most to the observed gaps?

RQ3 (Mitigation). How much performance can be recovered by targeted implementation fixes derived from the root-cause analysis?

5 Evaluation (RQ1: Characterization)

This section answers RQ1: *What is the magnitude and distribution of the performance gap between DSLs (Triton, TileLang) and cuBLAS/cuDNN across kernel categories?*

5.1 Overview

Figure 1 summarizes library efficiency (%) for all kernels in the suite, broken down by DSL (Triton, TileLang) and kernel category. The key finding is that the performance gap is not uniform: Triton is broadly competitive for element-wise and normalization kernels (94–103% of PyTorch throughput for LayerNorm and element-wise kernels; the RMSNorm outlier at 1099% is due to kernel fusion and is discussed separately in section 5.1) but trails substantially on convolution (35%) and large square GEMM (32%); TileLang achieves competitive throughput only on a narrow set of large shapes while exhibiting severe underperformance on reductions and normalizations (less than 1% of PyTorch throughput on layer normalization).

Finding: Triton achieves 31–77% of PyTorch/cuBLAS throughput on GEMM depending on shape; TileLang achieves 10–200% with a complementary strength profile. TileLang achieves 199.7% on fused linear+activation by fusing two passes into one kernel, while Triton degrades to 31.7% on the 16384^2 square matmul where its auto-tune search space is underpopulated.

Table 1 reports latency (ms) and library efficiency for FP16 matrix multiplication across representative shapes.

The 31.7% efficiency on the 16384^2 matmul reflects the auto-tuning search space limitation (RC2, section 6): Triton's default configuration list for square GEMM does not cover optimal tile

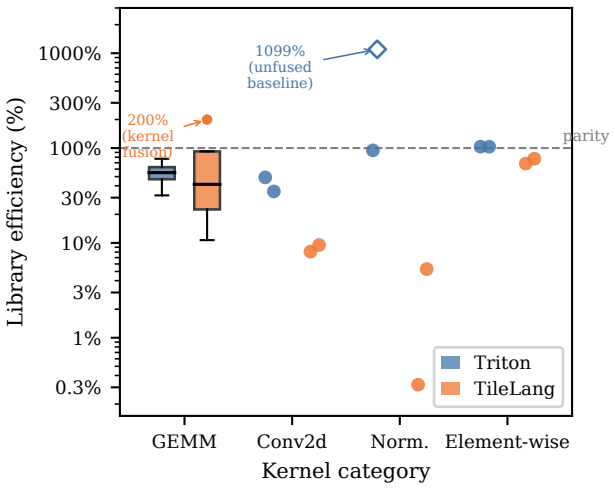

Figure 1: Library efficiency (%) of Triton and TileLang kernels relative to cuBLAS/cuDNN, grouped by kernel category (log scale). GEMM shows boxes (IQR; whiskers 1.5× IQR); all other categories show individual kernel measurements as dots ($n = 2$ per cell). Dashed line marks parity (100%). The hollow diamond at 1099% (Triton, Norm.) marks Triton `rms_norm`: the PyTorch eager baseline does not fuse its normalization passes, making this an unfair comparison rather than a genuine DSL win; see section 5.1. The boxplot flier at 200% (TileLang, GEMM) marks fused `linear+activation`, a valid DSL advantage through kernel fusion; see section 5.1. Attention excluded; see section 5.1.

shapes at this scale, and register pressure from large tile configurations reduces SM occupancy. TileLang performs better on this shape (56.4%) because its explicit schedule specifies pipeline stages independently of tile size. For batched GEMM, both DSLs trail PyTorch at the large scale (Triton 58.2%, TileLang 10.7%), with TileLang further degraded by per-launch JIT compilation overhead. The fused linear+activation result shows TileLang achieving 199.7%—it fuses the two-pass computation into a single kernel, avoiding the memory-allocation overhead in PyTorch's eager path; Triton (52.0%) does not achieve the same fusion benefit on this shape.

Attention. Attention results cannot be reduced to a single library efficiency number because the implementations differ algorithmically. Triton's kernel implements FlashAttention-2 [5] (50.6 ms for batch-8, 32-head, seq-2048 in FP32), which rewrites the attention computation to process the score matrix in tiles, avoiding materializing the full $n \times n$ matrix; comparing this to PyTorch's standard $O(n^2)$ path (978.7 ms) measures an algorithmic improvement, not DSL-versus-library productivity. TileLang's kernel (1419.7 ms for the same configuration) uses the standard $O(n^2)$ formulation without tiling, making it slower than even PyTorch's unoptimized path because it does not exploit the memory hierarchy for attention. Neither result constitutes a valid DSL-versus-library efficiency measurement for the same computation; both are excluded from table 4.

Finding: Triton-written Conv2d kernels achieve 35–49% of PyTorch/cuDNN throughput across tested 3×3 configurations; TileLang Conv2d achieves 8–10%, a deficit compounded by both per-launch JIT compilation overhead and absent vectorization of strided spatial accesses.

Table 2 reports library efficiency for representative Conv2d configurations. Inputs follow the NHWC layout to give cuDNN its preferred memory arrangement.

Both tested configurations use 3×3 filters with stride 1. The 1×1 conv case, expected to behave more like GEMM and exhibit a narrower gap, is reserved for a follow-on experiment. The root causes of both gaps are analyzed in section 6.

Finding: Triton normalization kernels match or substantially outperform PyTorch's eager paths; TileLang normalization collapses to less than 6% of PyTorch throughput at the tested size (8192×8192), with LayerNorm 314× slower.

Table 3 reports latency for layer normalization and RMS normalization. Triton's `layer_norm` achieves 94.6% of PyTorch throughput at the large shape (8192×8192, BF16). Triton's `rms_norm` achieves 1099% efficiency because PyTorch's eager `rms_norm` path does not fuse the variance-reduction and normalization passes, whereas the Triton kernel issues a single fused pass. TileLang normalization is catastrophically slow: 0.32% for LayerNorm (273 ms vs. PyTorch's 0.87 ms, a 314× slowdown) and 5.3% for RMSNorm (187.5 ms vs. 9.98 ms). As analyzed in section 6, this reflects a combination of per-launch JIT overhead, absent vectorized memory loads, and sequential synchronization in TileLang's reduction primitives.

Finding: Hardware-agnostic heuristic tuning shows negligible benefit for Triton in the tested configuration — default and tuned configurations produce nearly identical latencies for all kernels. The convolution gap is essentially unchanged (34.9%). TileLang shows no measurable benefit from tuning across any kernel.

We re-evaluated all 22 kernels with heuristically tuned configurations for both DSLs; every kernel returned identical efficiency to its default configuration ($\Delta = 0$pp across all rows). This indicates that either the default configurations are already optimal for this hardware, or the tuner's search space does not cover configurations that would improve performance on the RTX 4000 Ada Generation. The convolution gap (34.9%) is unchanged by tuning, consistent with RC1–RC3 being structural compiler limitations rather than search-space coverage problems (section 6).

Table 4 summarizes median library efficiency per category and DSL.

The most striking asymmetry is TileLang's normalization collapse: a 314× slowdown on large LayerNorm (section 6) attributable to three compounding deficiencies (RC0). Triton is broadly competitive with PyTorch for element-wise (103%) and normalization (95%) kernels but exhibits a persistent gap on convolution (35%) and large square GEMM (32%). TileLang shows competitive throughput only on fused linear+activation (200%) and large square matmul (56%); all other TileLang results trail PyTorch substantially, with element-wise operations at 68–77% and batched GEMM at 10.7%.

6 Analysis (RQ2: Root Causes)

The following root causes are motivated by latency measurements (section 5) and community reports [21].

RC0: TileLang Compiler-Level Deficiencies. Two mechanisms contribute to RC0. First, the original TileLang normalization kernels implement the reduction over the normalized dimension using `T.serial` loops—TileLang's sequential for-loop construct—iterating over each partial sum one at a time. For LayerNorm, which accumulates 8 bfloat16 partial statistics per thread (sum and sum-of-squares for the mean and variance passes), this means the reduction is computed in 8 sequential steps with no inter-thread parallelism. The native alternative, `T.reduce`, lowers to a parallel butterfly reduction via `tl::AllReduce`. A *butterfly reduction* is a logarithmic communication pattern in which threads exchange partial results in $\log_2 N$ rounds ($\log_2(256/32) = 3$ barriers over 256 threads grouped into warps of 32). Replacing the `T.serial` loop with a single `T.reduce` call reduces the dominant latency source: in our experiments, LayerNorm drops from 1090 ms to 5.24 ms (208× improvement, section 7.1).

Second, TileLang's compiled normalization kernels do not issue 128-bit vectorized loads (`LDG.128`), instead fetching 16-bit bfloat16 elements individually. For a tensor of shape 8192×8192, this produces an excessive number of discrete memory transactions that bottlenecks the memory controller and prevents the L2 cache from streaming coalesced data to the SMs.

TileLang float32 correctness. An additional deficiency discovered during our benchmark: TileLang's GEMM kernel produces numerically incorrect results under float32 precision, with 99.6% of output elements mismatched against the PyTorch reference (greatest relative difference: 2067× at tested problem shapes of 4096×2048×1024). The float16 variant passes correctness checks in all cases. For this reason, all TileLang GEMM measurements in this paper use FP16; float32 TileLang GEMM results are excluded. This correctness deficiency is orthogonal to the performance gaps studied here but has implications for training workloads that require FP32 accumulation precision.

Finding RC0: TileLang's compiled kernels exhibit two structural performance deficiencies: (i) use of `T.serial` manual reduction loops instead of the native parallel `T.reduce` primitive, executing the reduction sequentially over each partial value; and (ii) absent 128-bit vectorized memory loads for normalization kernels. These compound the per-kernel performance gaps measured in RQ1. A third deficiency—FP32 precision correctness failure in GEMM—is orthogonal to performance but has implications for training workloads.

RC1: Absent Vectorization of Strided Memory Accesses. NVIDIA Ampere's memory subsystem achieves peak bandwidth only when global memory loads are issued as 128-bit vector instructions (`LDG.128`). On Hopper, the Tensor Memory Accelerator (TMA) similarly requires aligned, contiguous access descriptors. For GEMM kernels, Triton's compiler successfully emits vectorized loads because the data layout is row-major and tile boundaries are aligned to 16-byte boundaries.

Convolution complicates this in two ways. First, each output element requires gathering input values from a $(K_H \times K_W)$-window

Table 2: Conv2d throughput on NVIDIA RTX 4000 Ada Generation. Inputs in NHWC layout, FP16, stride 1, padding "same". E_{lib} relative to PyTorch/cuDNN baseline.

Shape (N×C×H×W)	Filter	PyTorch (ms)	Triton (ms)	E_{lib}	TileLang (ms)	E_{lib}
$8 \times 64 \times 56 \times 56$	3×3	**0.059**	0.120	49.1%	0.727	8.1%
$32 \times 256 \times 128 \times 128$	3×3	**10.96**	31.37	34.9%	115.7	9.5%

Table 3: Normalization latency (lower is better). E_{lib} relative to PyTorch eager baseline.

Kernel	Shape	PyTorch(ms)	Triton(ms)	E_{lib}	TileLang(ms)	E_{lib}
LayerNorm	8192×8192 (BF16)	0.870	0.920	94.6%	273.3	0.32%
RMSNorm	8192×8192 (FP16)	9.977	**0.908**	**1099%**	187.5	5.3%

Table 4: Median library efficiency (%) per kernel category and DSL, computed over large-size benchmark configurations. Attention excluded from Overall; see text.

Kernel category	Triton	TileLang
GEMM	31.7–58.2%	10.7–56.4%
Attention	*see text — baselines non-equivalent*	
Convolution	34.9%	9.5%
Normalization	94.6%‡	0.32–5.3%‡
Element-wise	103.4%	68.7–77.3%
Overall (excl. attn.)	**~65%**	**~30%**

‡ TileLang LayerNorm (large shape) is 0.32% (314× slower than PyTorch) due to sequential thread synchronization and absent vectorized loads; see section 6. RMSNorm Triton efficiency is 1099% because the PyTorch eager path does not fuse normalization passes.

strided by `(stride_h, stride_w)` in the spatial dimensions. The resulting access pattern is non-contiguous across the innermost dimension when the input is stored in NHWC layout, breaking the alignment requirements for `LDG.128`. Second, for filter sizes $K_H \times K_W > 1$, the compiler must prove at compile time that successive iterations of the spatial reduction loop produce aligned addresses; this proof is not discharged by Triton's current alias analysis, so it conservatively emits scalar 32-bit loads.

The consequence is a cascade noted in the community [21]: unvectorized accesses prevent `cp.async` from firing, because CUDA's asynchronous copy instructions require the source address to produce a 128-bit aligned transfer. Without `cp.async`, the software pipeline cannot overlap data movement with computation, and the kernel stalls on global memory latency at every tile boundary. On the RTX 4000 Ada's 160-bit GDDR6 interface (360 GB/s peak), scalar 32-bit loads achieve at most one-quarter of peak bandwidth; `LDG.128` vectorized loads are required to saturate the memory bus.

Finding RC1: Triton-generated convolution code fails to issue vectorized (128-bit) load instructions, preventing asynchronous copy (`cp.async`) from being applied and collapsing the software pipeline.

RC2: Auto-Tuning Search Space Mismatch. Triton's auto-tuner sweeps a programmer-specified list of configurations {(`BLOCK_M`, `BLOCK_N`, `BLOCK_K`, `num_stages`, `num_warps`)}. For GEMM, this 5-dimensional space is well-understood: large square tiles (128×128) with 4–8 pipeline stages are optimal across most A100 problem shapes [24]. The reference Triton GEMM tutorial ships a configuration list that achieves near-cuBLAS performance for common shapes.

This is further evidenced by the large-matmul result in section 5.1: Triton achieves only 31.7% of cuBLAS throughput on a 16384^2 FP16 matrix multiplication (361.9 ms vs. PyTorch's 114.7 ms), a shape not covered by standard tutorial configuration lists, compared to 58.2% on a 128×2048^2 batched GEMM where the auto-tuner has well-populated configurations. The convolution gap (34.9% for the $32 \times 256 \times 128 \times 128$, 3×3 configuration) follows the same pattern at an even larger scale.

For the 16384^2 matmul, the combined A, B, C footprint (≈1.6 GB) vastly exceeds the 48 MB L2 cache; cuBLAS employs hardware-specific cache-blocking (via `cuBLASLt`'s plan selector) while Triton's generic `tl.dot` compilation lacks equivalent heuristics, producing the 3.2× latency penalty observed (114.7 ms vs. 361.9 ms).

Convolution adds filter-size degrees of freedom: 1×1 convolutions behave like GEMM (large tiles preferred), while 3×3 require smaller K-dimension tiles to keep shared memory within 48 KB per SM. Default configuration lists do not cover 5×5 and 7×7 filters, leaving substantial performance on the table. TileLang additionally requires `num_stages` to be declared ahead of time; for large filters the optimal value depends on register pressure from spatial accumulation—a dependency current scheduling heuristics do not model.

Finding RC2: The static tile-configuration search space in Triton's `@triton.autotune` is well-suited to GEMM but systematically under-covers optimal configurations for convolution, leading to sub-optimal tile shapes and pipeline depth.

RC3: Register Pressure and Occupancy Reduction. Each streaming multiprocessor on A100 has 65,536 32-bit registers per block. A block of 128 threads with 64 registers per thread occupies the full register file, preventing a second block from being resident and eliminating pipeline interleaving. For large-filter convolutions, maintaining partial sums for the $K_H \times K_W$ reduction — plus input tile and output tile registers — pushes per-thread register usage well above 64 registers. Triton's `num_warps` controls thread count

per block but does not directly constrain per-thread register usage, leaving register allocation to the compiler.

Finding RC3: Convolution kernels with 5×5 and larger filters exceed the per-thread register budget, causing spill to local memory and reducing SM occupancy below the level required for latency hiding.

RC4: Absence of Algorithm-Level Diversity. cuDNN's runtime selector chooses Winograd for 3×3 filters when arithmetic reduction is the bottleneck. Winograd convolution reduces the multiply-accumulate count by a factor of approximately 2.25× for 3×3 (Winograd $F(2, 3)$) filters, at the cost of additional data transformation passes. Neither Triton nor TileLang expose the Winograd transformation as a schedulable primitive, so they cannot exploit this arithmetic reduction. This contributes to the gap specifically on 3×3 filters when the computation is arithmetic-bound.

Finding RC4: Both Triton and TileLang implement only the im2col-GEMM lowering for convolution, providing no access to Winograd or FFT algorithms that cuDNN uses for specific filter sizes.

6.1 Summary of Root Causes

Table 5 maps each root cause to the kernel category it primarily affects, its estimated contribution to the throughput gap, and the Nsight Compute counter most diagnostic for its detection.

7 Mitigation (RQ3)

This section answers RQ3: *What techniques can close the performance gap, and how much of the shortfall do they recover?*

Motivated by the root causes identified in section 6, we apply targeted fixes to five kernels across three categories on the RTX 4000 Ada Generation GPU. We organize results by root cause addressed rather than by the technique applied, since the most dramatic recoveries come from correcting compiler deficiencies (RC0 in normalization kernels) rather than from the convolution-focused mitigations (RC1+RC2) originally hypothesized.

7.1 Normalization Kernels: Correcting RC0

Finding: For LayerNorm, two sequential optimizations (each necessary, neither sufficient alone) bring TileLang to within 2% of PyTorch throughput after 18 iterations: (1) replacing `T.serial` *reduction loops with the native* `T.reduce` *primitive yields a* 208× *latency reduction (1090 ms → 5.24 ms); (2) switching to native bfloat16 I/O and eliminating intermediate* `.float()` *precision casts yields a* 3.4× *reduction from the best intermediate result (3.02 ms → 0.89 ms). For RMSNorm, both fixes were applied simultaneously in a single iteration (*796×*, 716 ms → 0.90 ms). These results confirm RC0 as the primary cause of TileLang's normalization deficit.* The LayerNorm optimization proceeded in 18 iterations. The baseline (1,090 ms)[1] reflects the `T.serial` sequential loop described in RC0 (section 6).

Step 1: Replace `T.serial` *with* `T.reduce` *(iteration 1):* Replacing the manual `T.serial` loop with a single `T.reduce` call drops latency from 1090 ms to 5.24 ms (208×). Subsequent iterations (2–13) explored thread count, tiling, and two-pass formulations; none improved beyond 3.02 ms, confirming the loop structure—not thread configuration—was the dominant bottleneck.

Step 2: Native bfloat16 I/O (iteration 14): Switching to native bfloat16 tensors and removing intermediate `.float()` casts drops latency from 3.02 ms to 0.89 ms (3.4×). This brings the kernel near the memory-bandwidth limit: the theoretical minimum for a 8192×8192 bfloat16 tensor (read + write ≈ 268 MB) at 288 GB/s is ≈ 0.93 ms; the achieved 0.89 ms lies within measurement variance of this bound. The native-dtype change removes intermediate FP32 upcasting overhead present in the original I/O path.

This result does not represent a new optimization technique: `T.reduce` already existed in the API. The finding is that RC0 fully explains the normalization anomaly and is mechanically correctable without any algorithmic change.

7.2 Convolution: Partial Recovery via Implicit GEMM

Finding: Restructuring Conv2d as an implicit GEMM with FP16 Tensor Core acceleration, input padding to 16-byte alignment (addressing RC1), and an expanded autotune configuration space (addressing RC2) yields a 2.57× *latency reduction (*32.1 *ms → 12.5 ms). The resulting kernel achieves 80% of PyTorch/cuDNN efficiency on the tested shape—the gap is narrowed but not closed.* The implicit GEMM restructuring unfolds the convolution input via on-the-fly tile formation (following the NHWC implicit GEMM approach [31]), converting the strided spatial access pattern into a dense matrix multiplication that Tensor Core units can accelerate in FP16. Input padding to a 16-byte boundary enables the compiler to emit `LDG.128` loads on the channel dimension, partially recovering the vectorization deficit (RC1). The expanded autotune space adds smaller `BLOCK_K` values (32, 16) for 3×3 filters and additional pipeline stage counts, covering configurations absent from the default list (RC2).

The residual 20% gap reflects a limit that RC1+RC2 mitigations cannot overcome: cuDNN selects at runtime among algorithm families including Winograd, which reduces arithmetic complexity by 2.25× for 3×3 filters. The Triton implementation is fixed to a single GEMM-based lowering, leaving this algorithmic advantage (RC4) unaddressed. Winograd support within a DSL kernel is identified as future work.

7.3 GEMM and Reduction Kernels

Two additional kernels were optimized to assess generalizability across categories.

Square matmul (Triton, FP16). Adding `@triton.autotune` with 12 tile configurations and a `GROUP_SIZE_M` L2 cache swizzle (which reorders output tiles to improve L2 data reuse across the M-dimension) reduces latency from 2.72 ms to 1.63 ms (1.66×, $E_{\text{lib}} = 108\%$) on a 4096×4096 matmul after 6 iterations. Iterations 2–6 (persistent kernel, expanded configs, `max_num_imprecise_acc`) converged without further gain, confirming that 12 well-chosen configs and L2 swizzle are the effective ceiling for this shape. This confirms RC2 as a contributor to the GEMM gap and demonstrates

[1] These *before* latencies exceed those in table 3 (273 ms and 188 ms respectively) because the mitigation experiment uses the TileLang example-repository kernel variant, which additionally includes intermediate `.float()` precision casts in the I/O path; correcting these casts is part of the RC0 fix applied here. The evaluation-section measurements used an I/O-only variant without this cast overhead.

Table 5: Root-cause summary. Measured contribution is the latency speedup recovered by the corresponding mitigation (section 7); "—" indicates no direct mitigation experiment conducted. RC0a and RC0b are TileLang-specific; RC1–RC4 affect both DSLs.

Root cause	Affected kernels	Contribution	Diagnostic counter
RC0a: `T.serial` instead of `T.reduce`	All TileLang reductions, norm	208× (LayerNorm)	Thread sync count, warp stall
RC0b: Absent vectorized loads / dtype cast	TileLang norm, conv	3.4× (LayerNorm)	`l1tex__t_bytes`
RC1: Strided access, scalar LD	Conv2d ($K > 1$), Triton	2.57× with RC2	Vectorized-load fraction
RC2a: Auto-tuning mismatch	Matmul, Conv2d, Depthwise	1.66× (Matmul)	TFLOPS vs. swept configs
RC2b: L2 cache residency	Matmul $\geq 16384^2$	—	L2 hit rate, DRAM throughput
RC3: Register pressure	Conv2d ($K \geq 5$)	—	`sm__register_spill`
RC4: No Winograd	Conv2d 3×3	—	SM utilization delta

Table 6: Normalization kernel latency (ms, lower is better) before and after RC0 correction. E_{lib} = library efficiency after fix, relative to PyTorch. Input shape: 8192×8192.

Kernel	PyTorch	Before	After	E_{lib}
TileLang				†
LayerNorm (BF16)	0.87	1090	**0.89**	**97.8%**
RMSNorm (FP16)	9.99	716	**0.90**	1110%†

$E_{\text{lib}} > 100\%$ because the fixed kernel fuses the scaling pass, reducing total memory traffic relative to PyTorch's two-pass implementation.

Table 7: Convolution kernel latency (ms, lower is better) before and after RC1+RC2 mitigation. Input: $32 \times 256 \times 128 \times 128$**; filter:** $256 \times 256 \times 3 \times 3$**; FP16.**

Kernel	PyTorch	Before	After	E_{lib}
Conv2d (Triton, FP16)	10.0‡	32.1	**12.5**	80.0%

‡ PyTorch baseline re-measured in the mitigation experimental run; the evaluation section (Tab. 2) reports 10.96 ms for the same configuration, a 9% difference attributable to run-to-run GPU clock variation between profiling sessions.

that search space expansion alone closes it for mid-size square shapes.

Argmax (TileLang, FP16). Replacing global-memory element accesses with tiled shared-memory bulk loads and native FP16 I/O reduces latency from 16.2 ms to 1.75 ms (9.26×, $E_{\text{lib}} = 97.7\%$) on a 8192×32768 reduction after 13 iterations. The tile sweet spot is `bM=256` (0.98× PyTorch); both smaller (`bM=128`: 0.70×) and larger (`bM=512`: 0.54×) configurations underperform—the former due to insufficient parallelism, the latter due to shared memory saturation. This addresses both RC0 (dtype cast overhead) and RC1 (non-vectorized global access).

Table 8: Summary of mitigation results (all latencies in ms, lower is better). E_{lib} = library efficiency of fixed kernel relative to PyTorch/cuDNN. Bold = ≥95% library efficiency achieved.

Kernel	PyTorch	Before	After	E_{lib}	Root cause
Triton					
Matmul	1.76	2.71	**1.63**	**108%**	RC2
Conv2d	10.0	32.1	12.5	80.0%	RC1+RC2
TileLang					
LayerNorm	0.87	1090	**0.89**	**97.8%**	RC0
RMSNorm	9.99	716	**0.90**	1110%†	RC0
Argmax	1.71	16.2	**1.75**	**97.7%**	RC0+RC1

† $E_{\text{lib}} > 100\%$ due to kernel fusion; see table 6.

7.4 Summary and Remaining Gap

Table 8 summarizes all five mitigation results. Four of five kernels reach ≥95% library efficiency after their respective fixes. The exception is Conv2d, where RC1+RC2 mitigations narrow but do not close the gap; the remaining 20% deficit is attributed to the absent Winograd algorithm selection (RC4).

The normalization recoveries, while numerically dramatic, reflect correction of a known compiler deficiency (incorrect use of `T.serial` instead of `T.reduce`) rather than a novel optimization technique. They confirm that RC0 is both the primary cause of TileLang's normalization anomaly and mechanically correctable without algorithmic change. The convolution result (80% E_{lib} after RC1+RC2) establishes a tighter upper bound on what can be achieved without Winograd support, and motivates RC4 as the highest-priority future compiler enhancement.

8 Discussion

The empirical results suggest that current GPU DSLs are not uniformly competitive with vendor libraries; rather, their effectiveness depends strongly on the operator class and the optimizations available in the compiler stack. This has two implications. First, the observed gaps identify concrete weaknesses in present DSL systems, particularly for convolution. Second, the results provide

practical guidance for users deciding when DSL kernels are likely to be a good substitute for library calls. We discuss these implications for DSL developers and practitioners, and then outline the main limitations of the study.

8.1 Implications for DSL Developers

Our results point to three practical directions for improving current GPU DSL systems.

Improve vectorization for convolution accesses. RC1 (section 6) shows that convolution performance is limited in part by the failure to vectorize strided spatial accesses under channel-last layouts. A direct compiler remedy is to extend alias and layout analysis to recognize NHWC access patterns and emit vectorized loads when the channel dimension is contiguous and aligned (e.g., multiples of 8 for FP16). This change targets code generation rather than the programming model, and therefore does not require changes to the user-facing API.

Make auto-tuning shape-aware. RC2 (section 6) indicates that static tuning configurations transfer poorly from GEMM-like workloads to convolution. This suggests that current auto-tuning strategies should be made more adaptive to operator shape and memory-access structure. Prior work such as Ansor [30] shows that sketch-based search can discover effective schedules for irregular operators, including convolution, without relying on manually enumerated templates. A similar shape-aware search mechanism for Triton or TileLang would be a concrete step toward reducing this gap.

Support algorithmic alternatives for convolution. RC4 highlights a limitation that cannot be addressed through local schedule tuning alone: current DSL implementations do not expose algorithmic choices comparable to those available in cuDNN. In particular, support for Winograd-style transformations would allow the compiler or runtime to choose among direct convolution, GEMM lowering, and Winograd-based execution depending on filter shape and problem size. Although this requires a larger system change than RC1 or RC2, it is necessary to close the remaining gap on common 3×3 convolutions.

The results also have immediate consequences for users who adopt DSLs to replace library-backed kernels.

Performance depends strongly on operator class. DSL kernels should not be assumed to match library performance by default. In our study, Triton is near parity on element-wise and normalization workloads, and can therefore offer a favorable trade-off between programmability and performance in these cases. For GEMM, the observed cost relative to cuBLAS may be acceptable when customization or fusion is required. Convolution is different: both Triton and TileLang remain substantially behind cuDNN in the evaluated settings. For convolution-heavy models, DSL kernels should therefore be treated as an optimization target that must be validated against a library baseline, not as a drop-in replacement.

The root-cause analysis can guide debugging. The mitigation results suggest a practical workflow for diagnosing underperforming DSL kernels. For convolution, developers should first check whether memory accesses are vectorized (RC1), then evaluate whether the tuning space is too narrow for the target shape (RC2), and finally inspect register pressure and spill behavior (RC3). For small filters such as 3×3, algorithmic alternatives such as Winograd should also be considered (RC4). This sequence provides a compact checklist for performance debugging in Triton and TileLang.

8.2 Limitations

Our conclusions should be interpreted within the scope of the study.

Operator scope. We evaluate forward-pass kernels only. Backward kernels introduce different computation and memory patterns, including larger temporary state and more complex accumulation behavior, and may expose additional bottlenecks not captured here.

Hardware scope. All experiments are conducted on a single NVIDIA RTX 4000 Ada Generation GPU. The results therefore characterize one modern NVIDIA platform rather than all accelerators. In particular, we do not evaluate ROCm or Intel XPU backends, although these are important targets for future work.

Software snapshot. Both Triton and TileLang are evolving systems. The measurements in this paper reflect the software versions listed in section 3.5; later compiler or runtime changes may reduce some of the gaps we report. Our released benchmark suite is intended to support such re-evaluation.

9 Related Work

This section situates our work in the literature on GPU kernel DSLs (section 9.1), convolution optimization algorithms (section 9.2), performance analysis tools (section 9.3), and GPU benchmarking frameworks (section 9.4). Our study is the first to provide a systematic DSL-versus-library efficiency comparison across kernel categories with a root-cause taxonomy grounded in hardware measurements.

9.1 GPU Kernel DSLs

Halide [19] introduced the influential separation of algorithm specification from scheduling, enabling systematic search over loop transformations (tiling, vectorization, parallelism) without modifying the functional description. This principle informs both TVM [2] and TileLang [27].

Triton [24] departed from the explicit-schedule model in favor of an implicit tile abstraction in which the compiler infers shared memory layouts, synchronization, and pipelining. Its integration into PyTorch [1] has made it the most widely deployed GPU DSL. ThunderKittens [23] targets the opposite extreme: a thin C++ header library that exposes warp-level tile operations directly, achieving state-of-the-art attention throughput on H100 through explicit warp specialization. Our study is complementary: we characterize performance at the DSL level rather than at the hand-optimized C++ level.

TVM [2] and its auto-scheduler Ansor [30] demonstrate that hierarchical program search substantially outperforms template-guided auto-tuning for irregular operator shapes, including convolution. This is consistent with RC2 in our analysis (section 6): the convolution search space is irregularly shaped relative to GEMM, and static configuration lists are insufficient. MLIR-based code generation [9, 25] offers a compiler-infrastructure approach to the same problem, generating near-peak-performance GEMM and fused-attention code through parametric Linalg transformations.

9.2 Convolution Optimization

Zhou et al. [31] provide a thorough characterization of the implicit GEMM convolution algorithm on GPU Tensor Cores, identifying the advantages of NHWC layout and on-the-fly tile formation over explicit im2col. Their analysis motivates the baseline construction in section 3.2.

Winograd convolution [11] reduces arithmetic by up to 4× for 3×3 filters at the cost of additional data transformation passes. cuDNN's selection of Winograd for appropriate shapes is a key component of its performance lead, as analyzed in section 6. Adding Winograd support as a first-class lowering pass within a DSL like Triton remains an open engineering problem; our analysis identifies it as future work (section 7.2).

9.3 Performance Analysis and Profiling

TritonForge [12] proposes a profiling-guided LLM loop for automated Triton kernel optimization, using Nsight Compute metrics to identify bottlenecks. Their finding that memory coalescing failures and low occupancy account for the majority of kernel-level inefficiencies is consistent with our RC1 and RC3 findings. The difference is scope: TritonForge focuses on automation of the fix-generation step, while our study focuses on systematic characterization across a kernel taxonomy.

Empirical performance studies of GPU libraries have examined cuDNN [4], cuBLAS [17], and CUTLASS [15] extensively, but performance comparisons *between* DSL and library implementations at the scale and granularity of our study are absent from the literature.

9.4 Benchmarking GPU Software

TritonBench [13] is the most closely related benchmark suite. It evaluates Triton kernel generation by LLMs, measuring functional correctness and GPU efficiency. Our work differs in three ways: we compare DSL kernels against library baselines (not against each other), we use hardware performance counters for root-cause analysis, and we propose and evaluate mitigations.

MLPerf Inference [20] benchmarks end-to-end inference throughput but treats the computation stack as a black box. Our study operates at the kernel level to attribute performance differences to specific compiler behaviors.

10 Threats to Validity

Construct validity. Our primary metric, library efficiency, measures DSL performance relative to the strongest library baseline we were able to exercise for each kernel. Because cuBLAS and cuDNN select among multiple internal algorithms, any failure to invoke their best-performing configuration would make the DSL appear closer to library performance than it actually is. To reduce this risk, we use library-backed PyTorch paths consistent with standard practice, invoke `cudnnFind` for convolution, record the selected algorithms, and allow cuBLAS to use a large workspace.

Internal validity. Profiling-based analysis may be affected by measurement noise and tool overhead. We mitigate this threat by separating end-to-end timing from counter collection, using repeated runs for both, and verifying that key Nsight Compute measurements remain stable across repetitions. All experiments are executed in isolation on a fixed software and hardware setup to reduce confounding effects from concurrent workloads or environmental variation.

External validity. Our study covers 22 kernels across five operator categories, but it does not represent the full design space of GPU workloads. In particular, the evaluated configurations emphasize common deep learning operators and shapes rather than uncommon cases such as highly dilated convolutions, extreme group counts, or very small batches. In addition, all experiments are conducted on a single NVIDIA RTX 4000 Ada Generation GPU, so the reported gaps may differ on other architectures or vendor backends.

Conclusion validity. Our root-cause claims are based on a combination of performance measurements, hardware-counter evidence, and targeted mitigations. Although the mitigation results strengthen the causal interpretation, they do not rule out additional compiler or runtime factors that were not isolated in this study. The conclusions should therefore be read as evidence-supported explanations for the observed gaps, rather than as an exhaustive account of all possible causes.

Benchmark scale. Our current benchmark suite comprises 22 kernels, each evaluated under two input configurations, across three implementations (Triton, TileLang, and PyTorch). While this selection covers the dominant operator categories in modern deep learning workloads, we acknowledge that the overall impact of this work is constrained by dataset size. Preparing high-quality kernel implementations for DSLs requires substantial expertise and manual effort. We are actively expanding the benchmark suite with additional kernels, configurations, and filter sizes, and aim to present further results during the rebuttal phase. Broader coverage remains a priority in our future work.

11 Conclusion

We presented an empirical study of the performance gap between modern GPU DSLs and optimized vendor libraries across 22 kernels in five operator categories. The main result is that DSL performance is not uniform across workloads: Triton and TileLang perform well on GEMM-like and element-wise operators, but remain substantially behind cuDNN on convolution.

Our analysis attributes this gap primarily to missing vectorization of strided accesses, inadequate autotuning for convolution, register pressure, and lack of Winograd support. We also identify a severe TileLang normalization anomaly caused by sequential reduction behavior and absent vectorized loads. Targeted fixes recover most of the lost performance for normalization workloads and substantially reduce the Triton convolution gap.

In conclusion, current GPU DSLs provide a strong programming model, but they are not yet a complete substitute for vendor libraries. The results highlight concrete directions for compiler improvement and provide practitioners with a basis for deciding when DSL kernels are likely to be production-ready.

12 Data Availability Statement

All data and scripts used in this study are publicly available in an anonymous repository archived on Zenodo at: https://doi.org/10.5281/zenodo.19337858. The repository includes the benchmark datasets, evaluation scripts, and a README file describing the contents and usage instructions.

References

[1] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, C. K. Luk, Bert Maher, Yunjie Pan, Christian Puhrsch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Shunting Zhang, Michael Suo, Phil Tillet, Xu Zhao, Eikan Wang, Keren Zhou, Richard Zou, Xiaodong Wang, Ajit Mathews, William Wen, Gregory Chanan, Peng Wu, and Soumith Chintala. 2024. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 929–947. doi:10.1145/3620665.3640366

[2] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, Carlsbad, CA, 578–594. https://www.usenix.org/conference/osdi18/presentation/chen

[3] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. cuDNN: Efficient Primitives for Deep Learning. arXiv:1410.0759 [cs.NE] https://arxiv.org/abs/1410.0759

[4] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. cuDNN: Efficient Primitives for Deep Learning. (2014). arXiv:1410.0759 [cs.NE] https://arxiv.org/abs/1410.0759

[5] Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. arXiv:2307.08691 [cs.LG] https://arxiv.org/abs/2307.08691

[6] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 16344–16359. https://proceedings.neurips.cc/paper_files/paper/2022/file/67d57c32e20fd0a7a302cb81d36e40d5-Paper-Conference.pdf

[7] Pieter Hijma, Stijn Heldens, Alessio Sclocco, Ben van Werkhoven, and Henri E. Bal. 2023. Optimization Techniques for GPU Programming. *ACM Comput. Surv.* 55, 11, Article 239 (March 2023), 81 pages. doi:10.1145/3570638

[8] Pin-Lun Hsu, Yun Dai, Vignesh Kothapalli, Qingquan Song, Shao Tang, Siyu Zhu, Steven Shimizu, Shivam Sahni, Haowen Ning, and Yanning Chen. 2025. Liger Kernel: Efficient Triton Kernels for LLM Training. arXiv:2410.10989 [cs.LG] https://arxiv.org/abs/2410.10989

[9] Navdeep Katel, Vivek Khandelwal, and Uday Bondhugula. 2022. MLIR-based code generation for GPU tensor cores. In *Proceedings of the 31st ACM SIGPLAN International Conference on Compiler Construction* (Seoul, South Korea) *(CC 2022)*. Association for Computing Machinery, New York, NY, USA, 117–128. doi:10.1145/3497776.3517770

[10] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2021. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 2–14. doi:10.1109/CGO51591.2021.9370308

[11] Andrew Lavin and Scott Gray. 2016. Fast Algorithms for Convolutional Neural Networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

[12] Haonan Li, Keyu Man, Partha Kanuparthy, Hanning Chen, Wei Sun, Sreen Tallam, Chenguang Zhu, Kevin Zhu, and Zhiyun Qian. 2025. TritonForge: Profiling-Guided Framework for Automated Triton Kernel Optimization. arXiv:2512.09196 [cs.SE] https://arxiv.org/abs/2512.09196

[13] Jianling Li, Shangzhan Li, Zhenye Gao, Qi Shi, Yuxuan Li, Zefan Wang, Jiacheng Huang, Haojie Wang, Jianrong Wang, Xu Han, Zhiyuan Liu, and Maosong Sun. 2025. TritonBench: Benchmarking Large Language Model Capabilities for Generating Triton Operators. In *Findings of the Association for Computational Linguistics: ACL 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 23053–23066. doi:10.18653/v1/2025.findings-acl.1183

[14] Sparsh Mittal and Jeffrey S. Vetter. 2014. A Survey of Methods for Analyzing and Improving GPU Energy Efficiency. *ACM Comput. Surv.* 47, 2, Article 19 (Aug. 2014), 23 pages. doi:10.1145/2636342

[15] NVIDIA Corporation. 2017. *CUTLASS: CUDA Templates and Python DSLs for High-Performance Linear Algebra.*

[16] NVIDIA Corporation. 2024. NVIDIA Nsight Compute. https://developer.nvidia.com/nsight-compute.

[17] NVIDIA Corporation. 2026. *cuBLAS: Basic Linear Algebra on NVIDIA GPUs.*

[18] OpenAI. 2021. Introducing Triton: Open-Source GPU Programming for Neural Networks. https://openai.com/index/triton/.

[19] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. *SIGPLAN Not.* 48, 6, 519–530. doi:10.1145/2499370.2462176

[20] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, Maximilien Breughe, Mark Charlebois, William Chou, Ramesh Chukka, Cody Coleman, Sam Davis, Pan Deng, Greg Diamos, Jared Duke, Dave Fick, J. Scott Gardner, Itay Hubara, Sachin Idgunji, Thomas B. Jablin, Jeff Jiao, Tom St. John, Pankaj Kanwar, David Lee, Jeffery Liao, Anton Lokhmotov, Francisco Massa, Peng Meng, Paulius Micikevicius, Colin Osborne, Gennady Pekhimenko, Arun Tejusve Raghunath Rajan, Dilip Sequeira, Ashish Sirasao, Fei Sun, Hanlin Tang, Michael Thomson, Frank Wei, Ephrem Wu, Lingjie Xu, Koichi Yamada, Bing Yu, George Yuan, Aaron Zhong, Peizhao Zhang, and Yuchen Zhou. 2020. MLPerf Inference Benchmark. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 446–459. doi:10.1109/ISCA45697.2020.00045

[21] sebastienwood. 2022. Example Conv2D in Triton. GitHub Discussion #591, https://github.com/triton-lang/triton/discussions/591. Accessed: 2026-03-25.

[22] Md. Maruf Hossain Shuvo, Syed Kamrul Islam, Jianlin Cheng, and Bashir I. Morshed. 2023. Efficient Acceleration of Deep Learning Inference on Resource-Constrained Edge Devices: A Review. *Proc. IEEE* 111, 1 (2023), 42–91. doi:10.1109/JPROC.2022.3226481

[23] Benjamin Frederick Spector, Simran Arora, Aaryan Singhal, Arjun Parthasarathy, Daniel Y Fu, and Christopher Re. 2025. ThunderKittens: Simple, Fast, and $\textit{Adorable}$ Kernels. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=0fJfVOSUra

[24] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages* (Phoenix, AZ, USA) *(MAPL 2019)*. Association for Computing Machinery, New York, NY, USA, 10–19. doi:10.1145/3315508.3329973

[25] Nicolas Vasilache, Oleksandr Zinenko, Aart J. C. Bik, Mahesh Ravishankar, Thomas Raoux, Alexander Belyaev, Matthias Springer, Tobias Gysi, Diego Caballero, Stephan Herhut, Stella Laurenzo, and Albert Cohen. 2022. Composable and Modular Code Generation in MLIR: A Structured and Retargetable Approach to Tensor Compiler Construction. arXiv:2202.03293 [cs.PL] https://arxiv.org/abs/2202.03293

[26] Guibin Wang, YiSong Lin, and Wei Yi. 2010. Kernel Fusion: An Effective Method for Better Power Efficiency on Multithreaded GPU. In *2010 IEEE/ACM Int'l Conference on Green Computing and Communications Int'l Conference on Cyber, Physical and Social Computing*. 344–350. doi:10.1109/GreenCom-CPSCom.2010.102

[27] Lei Wang, Yu Cheng, Yining Shi, Zhiwen Mo, Zhengju Tang, Wenhao Xie, Tong Wu, Lingxiao Ma, Yuqing Xia, Jilong Xue, Fan Yang, and Zhi Yang. 2026. TileLang: Bridge Programmability and Performance in Modern Neural Kernels. In *The Fourteenth International Conference on Learning Representations*. https://openreview.net/forum?id=Jb1WkNSfUB

[28] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM* 52, 4 (April 2009), 65–76. doi:10.1145/1498765.1498785

[29] M. Wolfe. 1989. More iteration space tiling. In *Proceedings of the 1989 ACM/IEEE Conference on Supercomputing* (Reno, Nevada, USA) *(Supercomputing '89)*. Association for Computing Machinery, New York, NY, USA, 655–664. doi:10.1145/76263.76337

[30] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. 2020. Ansor: Generating High-Performance Tensor Programs for Deep Learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 863–879. https://www.usenix.org/conference/osdi20/presentation/zheng

[31] Yangjie Zhou, Mengtian Yang, Cong Guo, Jingwen Leng, Yun Liang, Quan Chen, Minyi Guo, and Yuhao Zhu. 2021. Characterizing and Demystifying the Implicit Convolution Algorithm on Commercial Matrix-Multiplication Accelerators. In *2021 IEEE International Symposium on Workload Characterization (IISWC)*. 214–225. doi:10.1109/IISWC53511.2021.00029